RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ D'ALGER 1
FACULTÉ DES SCIENCES

Compte Rendu

par:
Ouarezki Oussama Abde Rahim

Année Universitaire 2024-2025

Compte rendu

Ouarezki Oussama Abde Rahim G3

Introduction

In this report, we will explore the application of linear regression models to analyze and predict real-world outcomes. First, we will use simple linear regression to examine how years of professional experience impact employee salaries. Then, we will apply multiple linear regression to analyze how various factors, such as R&D spending, administration costs, marketing expenses, and location, affect the profit of startups. The goal of this report is to demonstrate how these regression models can provide valuable insights and aid in decision-making.

1 Simple linear regression

> Question 1
>
> Lisez l'ensemble de données *SalaryData.csv* et affichez les premières lignes pour mieux comprendre sa structure.

We imported the library *pandas* it's necessary for exploring and manging dataframes, and we used that function $pd.read_csv(")$ to store the csv file in variable called *data*, we explored the data using the methode $head()$ which show the first 5 rows of the dataframe.

```
import pandas as pd
import numpy as np

data=pd.read_csv("Salary_Data.csv")
data.head()
```

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0

	YearsExperience	Salary
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

Question 2

Séparez les variables explicatives (X) et la variable à prédire (y).

it's pretty straight forward we splitted the data into two parts:

- **indepadant variable** which is X
- **depadant variable** which is the Y in this case

```
X=data[['YearsExperience']]
Y=data['Salary']
```

Question 3

Divisez le dataset en Training set et Test set avec la fonction *traintestsplit* :
- L'ensemble de test doit représenter (33%) du dataset total.
- Utilisez random_state = 0 pour garantir des résultats reproductibles.

To use $train_test_split()$ we need to import the library *sklearn*

```
from sklearn.model_selection import train_test_split

X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.33,random_state=0)
```

Question 4

Construire le modèle de régression linéaire :
- Importez la classe LinearRegression de la bibliothèque sklearn.
- Entraînez le modèle sur l'ensemble d'entraînement.

We began by importing the LinearRegression model from scikit-learn and storing it in a variable called *model*. Next, we used the $fit()$ method to train the model on our data by providing the training dataset as input.

```
from sklearn.linear_model import LinearRegression
model=LinearRegression()

model.fit(X_train,Y_train)
```

```
LinearRegression()
```

> Question 5
>
> Utilisez le modèle que vous avez créé pour faire des prédictions sur l'ensemble de test. Affichez ensuite les valeurs prédites pour les comparer aux valeurs salariales réelles.

```
Y_pred=model.predict(X_test)

comp = pd.DataFrame({'real values': Y_test,
'predictedvalues': Y_pred,'residu': abs(Y_test-Y_pred)})

comp
```

	real values	predictedvalues	residu
2	37731.0	40835.105909	3104.105909
28	122391.0	123079.399408	688.399408
13	57081.0	65134.556261	8053.556261
10	63218.0	63265.367772	47.367772
26	116969.0	115602.645454	1366.354546
24	109431.0	108125.891499	1305.108501
27	112635.0	116537.239698	3902.239698
11	55794.0	64199.962017	8405.962017
17	83088.0	76349.687193	6738.312807
22	101302.0	100649.137545	652.862455

As we can see that the diffrence between the real values and predicted values are relatively little bit small, so we can conclude that our model is good enough.

> Question 6
>
> Estimez le salaire d'une personne ayant 15 ans d'expérience.

To estimate salary of someone who had 15 years of exprecience we first have to create 2D numpy arrays because the model need 2D array to work, then we use the methode *model.predict()* and put the Values we want to predict as input to this method, and then we just display it.

```
salary15=np.array([[15]])
estimated_salary15=model.predict(salary15)

print('Estimated salary for someone who had 15 years of expreciene is: ',estimated_salary15)
```

```
Estimated salary for someone who had 15 years of expreciene is:  [167005.32889087]
```

```
C:\Users\AL-Athir\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:4
  warnings.warn(
```

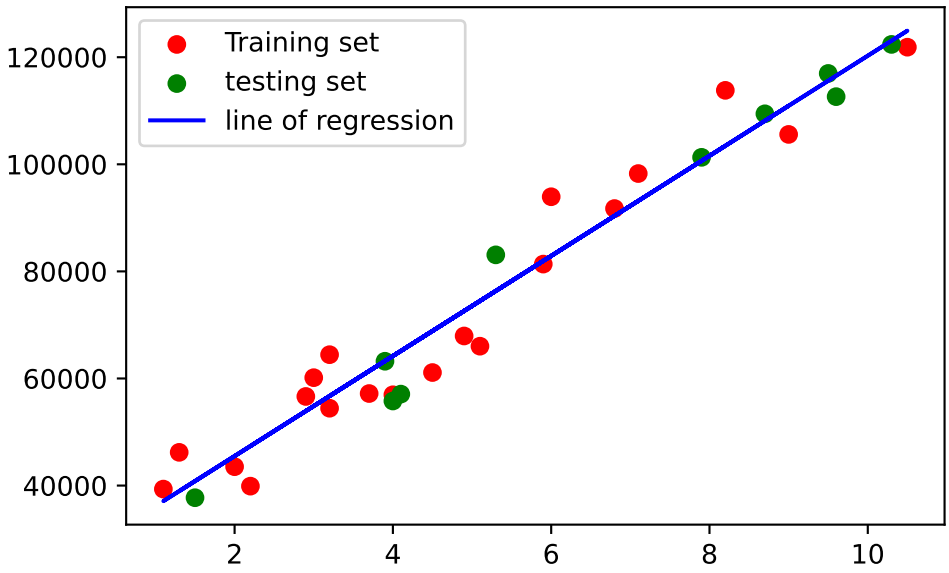
> Question 7
>
> Créez un graphique scatter pour visualiser la relation entre l'expérience et le salaire :
> - En rouge, les points de l'ensemble de test.
> - En vert, les points de l'ensemble d'entraînement.
> - Tracez la droite de régression correspondant à vos prédictions.

To plot our data and our line of regression we just need to import the library *matplotlib* and use it:
- for the training set we use for X:X_train and for Y:Y_train
- for the testing set we use for X:X_test and for Y:Y_test
- and for the line of regression X:X_train and for Y the predicted values of X_train.

And you are maybe asking why we used the training data to plot the regression line why we didn't use the testing data the answer is because we train the model with those data so those data are the ones who made the model create that line of regression

```
import matplotlib.pyplot as plt

plt.scatter(X_train,Y_train,color='red',label='Training set')
plt.scatter(X_test,Y_test,color='green',label='testing set')
plt.plot(X_train,model.predict(X_train),color='blue',label='line of regression')
plt.legend()
plt.show()
```


Training set
testing set
line of regression
120000
100000
80000
60000
40000
2
4
6
8
10

2 multiple linear regression

> Question 1
>
> Chargez l'ensemble de données Startups.csv, définissez X et y et encodez les variables catégoriques avec OneHotEncoder(drop='first') pour éviter la colinéarité.

As we can see that our dataframe have 5 columns 'R&D Spend', 'Administration', 'Marketing Spend', 'State', 'Profit' and the target variable is $Profit$.

```
df=pd.read_csv('Startups.csv')
df.head()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

To encode our data with $OneHotencoder$ we have first to import the library $sklearn$, and this case we will use $OneHotEncoder$ with $drop =' first'$ to prevent multicollinearity in regression models. **Original Categories** We have three categories: - California - Florida - New York

One-Hot Encoding Without Dropping If we encode the categories without dropping any, the resulting table is:

Category	California	Florida	New York
California	1	0	0
Florida	0	1	0
New York	0	0	1
Florida	0	1	0
California	1	0	0

One-Hot Encoding With drop When applying `drop='first'`, the first category (*California*) is dropped. The resulting table is:

Category	Florida	New York
California	0	0
Florida	1	0
New York	0	1
Florida	1	0
California	0	0

so when both are equal to zero $->$ that indicates that is California

```
X=df[['R&D Spend','Administration','Marketing Spend','State']]
Y=df['Profit']

from sklearn.preprocessing import OneHotEncoder

onehotencoder = OneHotEncoder(sparse_output=False,drop='first')
encoded = onehotencoder.fit_transform(df[['State']])
encoded = pd.DataFrame(encoded, columns=onehotencoder.get_feature_names_out(['State']))

X_encoded = pd.concat([X.drop(columns=['State']), encoded], axis=1)
```

The $sparse_output = False$ parameter in $OneHotEncoder$ ensures the output is a **dense NumPy array*- instead of a memory-efficient sparse matrix.
then we fit_transform our data and then name the columns and then drop the intial column $State$ and replaced by the new two Columns $State_Florida$ and $State_New_York$.

> Question 2
>
> Divisez le dataset en Training set et Test set puis utilisez LinearRegression pour entraîner le modèle sur X_train, y_train.

It's the same as we did before

```
X_train, X_test, Y_train, Y_test = train_test_split(X_encoded, Y, test_size=0.33, random_sta



model2 = LinearRegression()
model2.fit(X_train, Y_train)
```

```
LinearRegression()
```

Question 3

Estimez le profit d'une startup ayant des dépenses de 130000 en R&D, de 140000 en administration et de 300000 en marketing, ainsi qu'une localisation à New York.

```
X_encoded.columns
```

```
Index(['R&D Spend', 'Administration', 'Marketing Spend', 'State_Florida',
       'State_New York'],
      dtype='object')
```

as we can see the order here is:
1. R&D Spend
2. Administration
3. Marketing SpendProfit
4. State_Florida
5. State_New York'

so our vector have to respect this order.

```
new_startup = [[130000, 140000, 300000, 0, 1]]

predicted_profit = model2.predict(new_startup)


print(f"Estimated Profit: {predicted_profit[0]:.2f}")
```

```
Estimated Profit: 161753.13
```

```
C:\Users\AL-Athir\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:4
  warnings.warn(
```

Conclusion

In this session, we explored how linear regression can be used to analyze data and make predictions in practical scenarios:

1. Simple Linear Regression

We studied how an employee's experience influences their salary. Using a simple regression model, we identified a clear relationship and made predictions, like estimating the salary for someone with 15 years of experience. This showed us how straightforward models can provide valuable insights into real-world data.

2. Multiple Linear Regression

We expanded our understanding by analyzing how different factors, such as R&D, administration, marketing spending, and location, impact the profit of startups. By encoding categorical variables and training a regression model, we could predict the profit of a startup based on its spending and location, making the model highly applicable for business decision-making.

Key Takeaways:

- Prepare and split data effectively for model training and testing.
- Handle categorical variables for better model performance.
- Use regression models to understand and predict outcomes.
- Visualize data and model results to make them easier to interpret.

Overall, we saw how linear regression, both simple and multiple, can be powerful tools for analyzing data and making informed predictions in various fields.